

Spring Boot REST API

Pasted code.java

File

explain in details with example =

This is a very good list of Spring Boot REST API interview questions for a **3-5 years Java Backend Developer**. I also noticed a few answers are incomplete or slightly inaccurate (especially **PATCH idempotency**). I'll explain each one in an interview-friendly way with examples.

I'll start with the first set (Q1-Q5) in detail.

Pasted code

Q1. What is REST and why is it used?

Definition

REST stands for **Representational State Transfer**.

It is an **architectural style** used to build web services where the client and server communicate over HTTP.

REST exposes **resources** through URLs.

Example:

User Resource

```
/users
/users/10
/orders/20
/products/5
```

Instead of calling methods like

```
getUser()
saveUser()
deleteUser()
```

REST uses HTTP methods.

HTTP Method	Operation
GET	Read
POST	Create
PUT	Update
PATCH	Partial Update
DELETE	Delete

Why REST is used

REST is popular because it is

- Stateless
- Lightweight
- Scalable
- Easy to consume
- Platform independent
- Uses JSON (easy to understand)

Example

Suppose we have an Employee API.

Get employee

```
GET /employees/101
```

Response

```
JSON
{
  "id": 101,
  "name": "Ajay",
  "salary": 60000
}
```

Create employee

```
POST /employees
```

Request

```
JSON
{
  "name": "Ajay",
  "salary": 60000
}
```

Update employee

```
PUT /employees/101
```

Delete employee

```
DELETE /employees/101
```

Interview Answer (2 minutes)

REST stands for Representational State Transfer. It is an architectural style used to build web services. REST exposes resources using URLs and uses HTTP methods like GET, POST, PUT, PATCH and DELETE to perform CRUD operations. REST is stateless, scalable, lightweight and generally exchanges data in JSON format, making it ideal for communication between frontend applications, mobile apps and microservices.

Q2. Difference between POST and PUT

This is one of the most common interview questions.

POST	PUT
Creates new resource	Updates existing resource
Not idempotent	Idempotent
Server usually generates ID	Client specifies resource ID
Multiple requests create multiple records	Multiple requests keep same result

POST Example

POST /employees

Request

```
JSON
{
  "name": "Ajay"
}
```

Server creates

Employee ID = 101

Response

```
JSON
{
  "id": 101,
  "name": "Ajay"
}
```

Send same request again

POST /employees

New employee

ID = 102

Another request

POST /employees

Another employee

ID =103

So POST is **NOT idempotent**.

PUT Example

PUT /employees/101

Request

```
JSON
{
  "id":101,
  "name":"Ajay",
  "salary":70000
}
```

Employee becomes

ID=101
Salary=70000

Run same request 100 times.

Employee is still

ID=101
Salary=70000

Nothing changes.

Therefore PUT **is idempotent**.

Spring Boot Example

```
Java
@PostMapping("/employees")
public Employee save(@RequestBody Employee emp){
    return employeeService.save(emp);
}
```

```
Java
@PutMapping("/employees/{id}")
public Employee update(@PathVariable Long id,
                      @RequestBody Employee emp){

    return employeeService.update(id, emp);
}
```

Interview Answer

POST is mainly used to create a new resource whereas PUT updates an existing resource. POST is not idempotent because repeated requests create multiple resources. PUT is idempotent because sending the same request multiple times results in the same resource state.

Q3. Difference between PUT and PATCH

Your file says:

PATCH is not idempotent.

That is **not always correct**.

Important Interview Point

PATCH **can be idempotent** depending on implementation.

HTTP specification does **not guarantee** PATCH is idempotent.

Many REST APIs implement PATCH in an idempotent way.

So don't simply say

PATCH is not idempotent.

Instead say

PATCH is generally used for partial updates and may or may not be idempotent depending on how it is implemented.

PUT

Replace entire object.

Existing Employee

```
JSON
{
  "id": 101,
  "name": "Ajay",
  "salary": 50000,
```

```
"city": "Pune"
}
```

PUT Request

```
JSON
{
  "id": 101,
  "name": "Ajay",
  "salary": 70000,
  "city": "Mumbai"
}
```

Entire object replaced.

PATCH

Only update changed fields.

```
PATCH /employees/101
```

Request

```
JSON
{
  "salary": 70000
}
```

Only salary changes.

Everything else remains same.

Spring Boot Example

```
Java
@PatchMapping("/{id}")
public Employee updateSalary(@PathVariable Long id,
                             @RequestBody Employee employee){

    return employeeService.update(id, employee);
}
```

Interview Answer

PUT replaces the complete resource whereas PATCH updates only the required fields. PUT requires the entire object, while PATCH only sends the fields that need modification. PATCH is preferred when only a few fields change because it reduces network payload.

Q4. What is an Idempotent API?

This question confuses many candidates.

Definition

Calling an API

```
1 time
10 times
100 times
```

should produce the **same final result**.

That API is idempotent.

Example

Delete employee.

```
DELETE /employees/10
```

First call

Employee deleted.

Second call

Employee already deleted.

Final result

Employee absent.

Still same result.

Therefore DELETE is idempotent.

PUT example

```
Salary=60000
```

Call PUT

```
Salary=70000
```

Again

```
Salary=70000
```

Again

Salary=70000

Still same.

POST example

POST /employees

Every call

Creates

101

102

103

104

Different result.

Not idempotent.

Interview Answer

An idempotent API produces the same final state even if the same request is executed multiple times. GET, PUT and DELETE are idempotent, whereas POST is generally not because each request may create a new resource.

Q5. Difference between @PathVariable and @RequestParam

Very common Spring Boot interview question.

@PathVariable

Reads value from URL path.

Example URL

GET /employees/10

Controller

```
Java
@GetMapping("/employees/{id}")
public Employee getEmployee(@PathVariable Long id){
    return service.findById(id);
}
```



```
}

```

Here

```
id =10

```

@RequestParam

Reads value from query parameter.

URL

```
GET /employees?id=10

```

Controller

```
Java
@GetMapping("/employees")
public Employee getEmployee(@RequestParam Long id){
    return service.findById(id);
}

```

Multiple parameters

```
GET /employees?page=1&size=10

```

```
Java
@GetMapping("/employees")
public List<Employee> getEmployees(
    @RequestParam int page,
    @RequestParam int size){
}

```

Comparison

@PathVariable	@RequestParam
Reads value from URL path	Reads value from query string
Mandatory in path	Can be optional
Used for resource identification	Used for filtering, sorting, pagination
/employees/10	/employees?id=10

Real-world examples

Get one employee

```
GET /employees/101
```

Use

```
Java
@PathVariable
```

Search employees

```
GET /employees?city=Pune
```

Use

```
Java
@RequestParam
```

Interview Answer

`@PathVariable` extracts values from the URI path and is mainly used to identify a specific resource, such as `/employees/101`. `@RequestParam` extracts values from the query string and is typically used for optional inputs like filtering, sorting, searching, or pagination, such as `/employees?page=1&size=10`.

Interview Tip

For a **4-year Java Spring Boot interview**, don't just define these concepts. Explain them with a small real-world example, like an Employee API, and mention where you've used them in your projects. That demonstrates practical experience rather than memorized definitions.

I can also explain **Q6-Q15** (HTTP status codes, exception handling, validation, JWT, API versioning, authentication vs authorization, and microservice communication) in the same detailed interview-oriented style.

yes continue

Great. Let's continue with **Q6-Q15** in the same interview style with **real examples, Spring Boot code, and interview tips**. These questions are frequently asked in Java/Spring Boot interviews for **3-5 years experience**.

Pasted code

Q6. What HTTP Status Codes have you used?

HTTP Status Codes tell the client whether the request was successful or failed.

Think of them like a delivery status:

-  Delivered Successfully
-  Wrong Address
-  Unauthorized
-  Server Failed

1. 200 OK

Everything is successful.

Used for

- GET
- Successful PUT
- Successful PATCH

Example

```
http
GET /employees/101
```

Response

```
JSON
HTTP/1.1 200 OK

{
  "id": 101,
  "name": "Ajay"
}
```

Spring Boot

```
Java
@GetMapping("/{id}")
public ResponseEntity<Employee> getEmployee(@PathVariable Long id){
    return ResponseEntity.ok(employeeService.findById(id));
}
```

2. 201 Created

Used when a new resource is created.

Example

```
http
POST /employees
```

Response

```
http
HTTP/1.1 201 Created
```

Spring Boot

```
Java
@PostMapping
public ResponseEntity<Employee> save(@RequestBody Employee employee){

    Employee saved = service.save(employee);

    return new ResponseEntity<>(saved,HttpStatus.CREATED);

}
```

3. 204 No Content

Operation successful

No response body required.

Mostly used in DELETE.

```
http
DELETE /employees/10
```

Response

```
http
204 No Content
```

Spring Boot

```
Java
@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(@PathVariable Long id){

    service.delete(id);

    return ResponseEntity.noContent().build();

}
```

4. 400 Bad Request

Client sent invalid data.

Example

```
JSON
{
  "email":"abc"
```

```
}

```

Email validation failed.

Response

```
http
400 Bad Request

```

5. 401 Unauthorized

User not logged in.

JWT missing.

```
Authorization Header Missing

```

Return

```
401 Unauthorized

```

6. 403 Forbidden

User logged in

But doesn't have permission.

Example

```
User Role = CUSTOMER
Trying to access
/admin

```

Return

```
403 Forbidden

```

7. 404 Not Found

Requested resource doesn't exist.

```
GET /employees/999

```

Employee not found.

Return

```
404 Not Found

```

8. 409 Conflict

Interviewers love this one.

Occurs when resource already exists.

Example

Trying to register

```
Email = ajay@gmail.com
```

Already exists.

Return

```
409 Conflict
```

9. 500 Internal Server Error

Unexpected server error.

Examples

- NullPointerException
- Database down
- Server crash

Return

```
500 Internal Server Error
```

Interview Answer

I commonly use 200 for successful GET requests, 201 for resource creation, 204 for successful deletion without a response body, 400 for validation failures, 401 when authentication is missing or invalid, 403 when access is denied, 404 when a resource isn't found, 409 for duplicate resource conflicts, and 500 for unexpected server-side errors.

Q7. When would you return 200, 201, 204, 400, 404, 500?

Interviewers want **real-world scenarios**, not just definitions.

Status	Scenario
200	Successfully fetched employee
201	New employee created
204	Employee deleted
400	Invalid request data

Status	Scenario
404	Employee ID not found
500	Database/server error

Example

GET /employee/101

→200

POST /employee

→201

DELETE /employee/101

→204

POST

Email invalid

→400

Employee not found

→404

Database crashed

→500

Q8. How do you handle exceptions in REST API?

Without exception handling

NullPointerException

Client gets

Whitelabel Error Page

Not professional.

Instead

Return

JSON

```
{
  "time": "2026-07-01",
  "status": 404,
  "message": "Employee Not Found"
}
```

Best Practice

Use

```
@ControllerAdvice
```

and

```
@ExceptionHandler
```

Custom Exception

```
Java
public class EmployeeNotFoundException extends RuntimeException{
    public EmployeeNotFoundException(String msg){
        super(msg);
    }
}
```

Global Handler

```
Java
@ControllerAdvice
public class GlobalExceptionHandler{
    @ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<String> handle(EmployeeNotFoundException ex){
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

Service

```
Java
Employee employee = repository.findById(id)
    .orElseThrow(() ->
        new EmployeeNotFoundException("Employee not found"));
```

Client receives

```
JSON
404

{
  "message": "Employee not found"
}
```

Interview Answer

I use centralized exception handling with `@ControllerAdvice` and `@ExceptionHandler`. Custom exceptions are thrown from the service layer and mapped to appropriate HTTP status codes. This provides consistent error responses and keeps controller code clean.

Q9. What is Global Exception Handling?

Instead of

Controller A

```
try
catch
```

Controller B

```
try
catch
```

Controller C

```
try
catch
```

Create

One Global Handler

@ControllerAdvice

Every controller uses it.

Advantages

- Clean code
- Centralized logic
- Consistent responses
- Easy maintenance

Q10. How do you validate request payload?

Spring Boot uses

Jakarta Bean Validation

Annotations

@NotNull

@NotBlank

@Email

@Pattern

@Min

@Max

DTO

```
Java
public class EmployeeDTO{

    @NotBlank
    private String name;

    @Email
    private String email;

    @Min(10000)
    private double salary;

}
```

Controller

```
Java
@PostMapping
public Employee save(

@Valid
@RequestBody EmployeeDTO dto){

}
```

Invalid Request

```
JSON
{
  "name": "",
  "email": "abc"
}
```

Automatically returns

400 Bad Request

Customize Validation Error

Handle

MethodArgumentNotValidException

inside

@ControllerAdvice

Interview Answer

I validate incoming request payloads using Jakarta Bean Validation annotations like `@NotBlank`, `@Email`, and `@Min`. I use `@Valid` in the controller, and if validation fails, Spring throws `MethodArgumentNotValidException`, which I handle globally to return user-friendly error messages.

Q11. What is Content Negotiation?

Suppose

One client wants

JSON

Another wants

XML

REST API can return both.

Client sends

Accept: application/json

or

Accept: application/xml

Spring chooses accordingly.

Controller

```
Java
@GetMapping(
    produces={
        MediaType.APPLICATION_JSON_VALUE,
        MediaType.APPLICATION_XML_VALUE
    })
```

Usually

Modern applications use

JSON

almost everywhere.

Interview Answer

Content negotiation allows the client to specify the desired response format using the **Accept** header. Spring Boot automatically selects the appropriate message converter to produce JSON, XML, or another supported format.

Q12. How do you version REST API?

Suppose

Current API

```
/employees
```

Later

New fields added.

Old clients break.

Solution

Versioning.

URI Versioning (Most Common)

```
/api/v1/employees
```

```
/api/v2/employees
```

Most companies use this.

Header Versioning

```
Accept-Version:2
```

Query Parameter

```
/employees?version=2
```

Interview Answer

The most common approach is URI versioning, such as `/api/v1/employees` and `/api/v2/employees`, because it's easy to understand and maintain. Header and query parameter versioning are also possible but are used less frequently.

Q13. Authentication vs Authorization

Very important interview question.

Authentication

Who are you?

Authorization

What can you access?

Example

Login

Username

Password

↓

Identity verified

↓

Authentication Complete

↓

Check Role

ADMIN

USER

MANAGER

↓

Authorization

Example

Ajay logs in.

Authentication

✓ Verified

Now tries

DELETE Employee

Role

USER

Delete allowed only for

ADMIN

Result

403 Forbidden

Interview Answer

Authentication verifies the user's identity using credentials such as username/password or JWT tokens. Authorization determines what the authenticated user is allowed to access based on roles or permissions.

Q14. How is JWT used in securing APIs?

This is one of the most frequently asked Spring Security questions.

Step 1: User Login

POST /login

Request

```
JSON
{
  "username": "ajay",
  "password": "123"
}
```

↓

Spring Security verifies credentials.

Step 2: Server Creates JWT

JWT consists of three parts:

Header

Payload

Signature

Example

xxxxx.yyyyy.zzzzz

Payload

```
JSON
{
  "sub": "Ajay",
  "role": "ADMIN",
}
```

```
"exp":1782000000
}
```

Step 3: Client Stores Token

Usually in:

- Browser memory
- Secure cookie
- Mobile app secure storage

Step 4: Client Sends JWT

```
Authorization: Bearer eyJhb...
```

Step 5: JWT Filter

Spring Security Filter

- Reads token
- Validates signature
- Checks expiration
- Loads user details
- Creates authentication object in the security context

Step 6: Controller Executes

If token is valid

```
200 OK
```

Else

```
401 Unauthorized
```

Interview Answer

After successful login, the server generates a signed JWT containing user information and roles. The client sends the token in the **Authorization: Bearer <token>** header for every request. A Spring Security filter validates the token, loads the user into the security context, and only then allows access to secured endpoints. Since JWT is stateless, the server doesn't need to maintain session data.

Q15. How do Microservices communicate through REST APIs?

Imagine three services:

Order Service

Inventory Service

Payment Service

Flow:



Example

Customer places an order.

1. Order Service receives request.
2. Calls Inventory Service to check stock.
3. If stock is available, calls Payment Service.
4. If payment succeeds, order is confirmed.

In Spring Boot, communication is typically done using:

- **RestTemplate** (legacy)
- **WebClient** (recommended for reactive/non-blocking)
- Declarative clients like **Feign Client** (very common in microservices)

For fault tolerance, combine these with tools like **Resilience4j** (Circuit Breaker, Retry, Timeout).

Example with **WebClient**:

```

Java
WebClient webClient = WebClient.create();

Employee employee = webClient.get()
    .uri("http://employee-service/employees/101")
    .retrieve()
    .bodyToMono(Employee.class)
    .block();
  
```

Interview Answer

In a microservices architecture, services communicate over HTTP using REST APIs. One service exposes endpoints, and another service consumes them using clients such as **WebClient** or Feign Client. To improve resilience, we use patterns like Circuit Breaker, Retry, and Timeouts with Resilience4j. Communication should be loosely coupled, and service discovery or an API Gateway is often used in production environments.

★ Interview Tips for a 4-Year Java Backend Developer

If the interviewer asks these questions, don't stop at definitions. Explain them with:

1. **The concept** (what it is).
2. **A real-world example** (Employee, Order, Payment, etc.).
3. **How you've used it** in your projects.

For example:

- "We used `@ControllerAdvice` to return consistent JSON error responses across all microservices."
- "We secured APIs using JWT and role-based authorization with Spring Security."
- "Our services communicated using REST APIs, and we used Resilience4j to handle downstream failures gracefully."

That combination shows practical experience, which interviewers value much more than memorized definitions.